Linux网络--Reactor模式层05 Connection回调与事件分发

> ⓘ 系列导航
>
> 上一篇：Linux网络--IO多路转接层04 LT与ET及epoll服务器实现
> 系列索引：Linux网络--IO多路转接与Reactor系列总览
> 下一篇：Linux网络--Reactor模式层06 OneThreadOneLoop与eventfd

- Connection、回调与事件分发
  - 1. 为什么有了 epoll 还需要 Reactor
  - 2. Reactor 中的职责划分
  - 3. Connection 保存一条连接的完整状态
    - 3.1 核心字段
    - 3.2 输入缓冲区为什么必须有
    - 3.3 输出缓冲区为什么必须有
    - 3.4 回调分工
  - 4. ConnectionFactory：不同连接有不同回调
  - 5. Accepter：把新连接纳入事件循环
    - 5.1 为什么 accept 要循环
    - 5.2 新连接不能直接读
  - 6. HandlerConnection：协议层与业务层
    - 6.1 接收路径
    - 6.2 协议解码循环
    - 6.3 为什么业务层不直接 send
  - 7. Sender：发送到 EAGAIN 或清空
  - 8. Reactor 管理全部 Connection
    - 8.1 映射关系
    - 8.2 添加连接
    - 8.3 修改事件
    - 8.4 删除连接
  - 9. Reactor 事件循环
  - 10. 板书：Reactor 的层次
    - 10.1 向下：事件从哪里来
    - 10.2 横向：怎样找到对象
    - 10.3 向上：状态怎样回到业务
  - 11. Connection 生命周期

    - 11.1 创建
    - 11.2 运行
    - 11.3 销毁
- 12. 缓冲区上限与背压
- 13. 业务处理不能阻塞事件循环

Connection、回调与事件分发

1. 为什么有了 epoll 还需要 Reactor

epoll 解决的是一类底层问题：

```
现在哪些 fd 发生了哪些事件？
```

服务器还需要解决另一类问题：

```
这个 fd 属于哪个连接？
这个连接收到半包时怎样保存数据？
处理完请求后响应放在哪里？
发送没发完时下一次应该关心什么事件？
连接关闭后怎样从所有管理结构中移除？
```

如果把这些逻辑全部写在 `epoll_wait` 后面的 `if/else` 里，程序最终会变成：

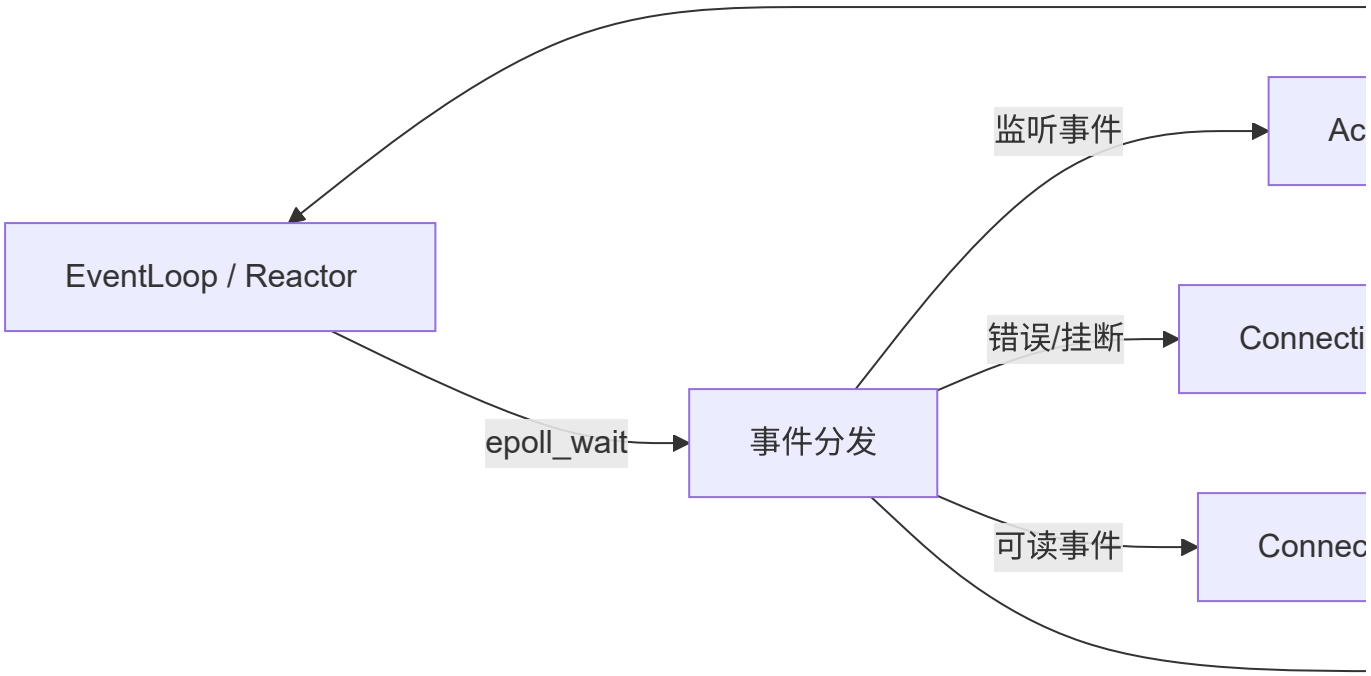
```
一个巨大的事件循环
  + 一组 fd 到 socket 的映射
  + 分散的请求缓冲区
  + 分散的发送偏移
  + 分散的关闭标记
  + 难以追踪的回调判断
```

Reactor 是一种事件驱动的组织方式。它让事件循环负责“等和分发”，让连接对象负责“保存状态”，让回调负责“推进下一步动作”。

```
epoll_wait
    -> 找到 Connection
```

```
    -> 根据事件调用 recver/sender/excepter
    -> 回调修改下一次关心的事件
```

Reactor 的核心对象关系可以画成：



2. Reactor 中的职责划分

一个最小可维护的 Reactor 可以划分为五层：

组件	主要职责
`Connection`	保存 fd、事件、缓冲区和回调
`Accepter`	处理监听 socket，创建新连接
`HandlerConnection`	协议解码、业务调用、编码响应
`Reactor / TcpServer`	管理所有连接并执行 epoll 事件循环
`EventLoop`	封装等待、唤醒和跨线程任务调度

这种划分不是让每个类只做一行代码，而是让状态变化有明确归属：

- epoll 事件到达后，先由 Reactor 找到连接；
- 连接自身决定应该调用哪个回调；
- 协议 Handler 决定字节怎样变成请求；

- 业务 Handler 决定请求产生什么响应；
- Reactor 根据连接的事件集合重新注册 epoll。

3. Connection 保存一条连接的完整状态

3.1 核心字段

```
class Connection;
class Reactor;

using func_t = std::function<void(Connection*)>;

class Connection {
public:
    Connection(int sockfd, uint32_t events, Reactor* reactor)
        : sockfd_(sockfd),
          events_(events),
          reactor_(reactor) {}

    void RegisterCallback(
        func_t recver,
        func_t sender,
        func_t excepter
    ) {
        recver_ = std::move(recver);
        sender_ = std::move(sender);
        excepter_ = std::move(excepter);
    }

private:
    int sockfd_;
    uint32_t events_;

    std::string in_buffer_;
    std::string out_buffer_;

    func_t recver_;
    func_t sender_;
    func_t excepter_;

    Reactor* reactor_;
    sockaddr_in client_{};
};
```

字段之间的关系：

```
sockfd_      内核中的网络连接
events_      当前希望 epoll 关心什么
in_buffer_   已经收到但还没有组成完整请求的字节
out_buffer_  已经生成但还没有完全发出的响应
callbacks    事件发生时推进哪条业务路径
reactor_     修改关注事件和注销连接
```

3.2 输入缓冲区为什么必须有

TCP 是字节流，不保留应用层消息边界。一次 `recv` 可能得到：

- 半个请求；
- 一个完整请求；
- 一个半请求；
- 多个完整请求。

因此不能把“一次 recv”直接当成“一条消息”。`in_buffer_` 的职责是持续保存尚未消费的字节，由协议解码器反复尝试提取完整请求。

3.3 输出缓冲区为什么必须有

`send` 也不保证一次写完。输出缓冲区保存：

- 已经生成但尚未写入内核发送缓冲区的内容；
- 非阻塞发送遇到 `EAGAIN` 时剩余的部分。

只有当输出缓冲区清空后，才可以关闭 `EPOLLOUT`。如果长期保留写事件，事件循环会不断报告可写。

3.4 回调分工

`recver_`：

- socket 可读时调用；
- 读取数据到输入缓冲区；
- 驱动协议解析；

- 必要时产生输出和启用写事件。

`sender_`：

- socket 可写时调用；
- 从输出缓冲区继续发送；
- 发送完后关闭写事件。

`excepter_`：

- 错误或挂断时调用；
- 从 epoll 和映射中移除连接；
- 关闭 socket；
- 安全销毁对象。

4. ConnectionFactory：不同连接有不同回调

监听 socket 没有业务请求，也没有普通发送路径；客户端连接需要完整的读、写、异常处理。可以用工厂创建两类对象。

```
class ConnectionFactory {
public:
    static Connection* BuildListenConnection(
        int listen_fd,
        func_t accept_cb,
        Reactor* reactor
    ) {
        auto* conn = new Connection(
            listen_fd,
            EPOLLIN,
            reactor
        );
        conn->RegisterCallback(accept_cb, nullptr, nullptr);
        return conn;
    }

    static Connection* BuildNormalConnection(
        int client_fd,
        func_t recver,
        func_t sender,
        func_t excepter,
```

```
        Reactor* reactor
    ) {
        auto* conn = new Connection(
            client_fd,
            EPOLLIN | EPOLLET,
            reactor
        );
        conn->RegisterCallback(recver, sender, excepter);
        return conn;
    }
};
```

监听连接和普通连接使用同一种 `Connection` 外壳，但回调不同。这样事件分发层不需要知道自己处理的是监听 fd 还是业务连接，只负责调用绑定的回调。

5. Accepter：把新连接纳入事件循环

5.1 为什么 accept 要循环

当监听 fd 使用 ET 时，一次可读事件可能表示 backlog 中已有多个待处理连接。只 accept 一次会让剩余连接等待下一次边沿。

```
class Accepter {
public:
    static void AcceptLoop(Connection* listen_conn) {
        for (;;) {
            sockaddr_in peer{};
            socklen_t len = sizeof(peer);

            int client_fd = accept(
                listen_conn->SockFd(),
                reinterpret_cast<sockaddr*>(&peer),
                &len
            );

            if (client_fd >= 0) {
                SetNonBlocking(client_fd);

                auto recver = std::bind(
                    &HandlerConnection::Recver,
                    std::placeholders::_1
                );
                auto sender = std::bind(
```

```
                    &HandlerConnection::Sender,
                    std::placeholders::_1
                );
                auto excepter = std::bind(
                    &HandlerConnection::Excepter,
                    std::placeholders::_1
                );

                Connection* client =
                    ConnectionFactory::BuildNormalConnection(
                        client_fd,
                        recver,
                        sender,
                        excepter,
                        listen_conn->ReactorPtr()
                    );

                client->SetClient(peer);
                listen_conn->ReactorPtr()->AddConnection(client);
                continue;
            }

            if (errno == EINTR) continue;
            if (errno == EAGAIN || errno == EWOULDBLOCK) break;

            // 真实错误
            break;
        }
    }
};
```

5.2 新连接不能直接读

`accept` 成功只说明完成 TCP 连接建立。客户端是否已经发送应用层数据，要由 epoll 再次判断。因此正确流程是：

```
accept
    -> 设置非阻塞
    -> 创建 Connection
    -> 注册 EPOLLIN
    -> 加入 Reactor
    -> 等待可读事件
    -> recver
```

不能因为刚建立连接就假设请求已经到达。

6. HandlerConnection：协议层与业务层

6.1 接收路径

接收函数先处理字节，不解释业务：

```
static void Recver(Connection* conn) {
    char buffer[4096];

    for (;;) {
        ssize_t n = recv(
            conn->SockFd(),
            buffer,
            sizeof(buffer),
            0
        );

        if (n > 0) {
            conn->AddInBuffer(std::string(buffer, n));
            continue;
        }
        if (n == 0) {
            conn->Exception();
            return;
        }
        if (errno == EINTR) continue;
        if (errno == EAGAIN || errno == EWOULDBLOCK) break;

        conn->Exception();
        return;
    }

    HandleRequest(conn);
}
```

步骤：

1. 将本次收到的所有字节追加到连接输入缓冲区；
2. 读到 `EAGAIN`，确认本轮排空；
3. 把输入缓冲区交给协议处理；

4. 协议处理可能产生一个或多个响应。

6.2 协议解码循环

```
static void HandleRequest(Connection* conn) {
    std::string message;

    while (Decode(conn->InBuffer(), &message)) {
        Request* request = factory.BuildRequest();
        if (!request->Deserialize(message)) {
            continue;
        }

        Response* response = calculate.Cal(request);

        std::string payload;
        response->Serialize(&payload);
        std::string frame = Encode(payload);

        conn->AddOutBuffer(frame);
    }

    if (!conn->OutBufferEmpty()) {
        Sender(conn);
    }
}
```

`Decode` 必须循环，因为一次输入缓冲区中可能包含多个完整报文。只有真正完整的报文才能从输入缓冲区消费。

完整链路是：

```
字节流
  -> 找消息边界
  -> Decode 得到一个完整报文
  -> Deserialize 得到请求对象
  -> 业务计算
  -> Serialize 得到响应字节
  -> Encode 添加消息边界
  -> 追加到输出缓冲区
```

6.3 为什么业务层不直接 send

业务层只负责产生响应字节。真正的发送由 `Sender` 和连接事件状态机负责。

这样可以把：

- 业务处理；
- TCP 流边界；
- 部分发送；
- EPOLLOUT 开关；
- 连接关闭；

分开管理。否则每个业务处理函数都要重复写一部分发送逻辑。

7. Sender：发送到 EAGAIN 或清空

```
static void Sender(Connection* conn) {
    std::string& output = conn->OutBuffer();

    while (!output.empty()) {
        ssize_t n = send(
            conn->SockFd(),
            output.data(),
            output.size(),
            MSG_NOSIGNAL
        );

        if (n > 0) {
            output.erase(0, static_cast<size_t>(n));
            continue;
        }
        if (n < 0 && errno == EINTR) continue;
        if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
            break;
        }

        conn->Exception();
        return;
    }

    bool need_write = !output.empty();
    conn->ReactorPtr()->EnableReadWrite(
```

```
        conn,
        true,
        need_write
    );
}
```

状态变化：

```
发送前：out_buffer 非空
        |
        v
send 一部分，删除已发送前缀
        |
        +-- out_buffer 清空 -> 下次只关心 EPOLLIN
        |
        +-- send 返回 EAGAIN -> 保留剩余 -> 关心 EPOLLIN | EPOLLOUT
```

`send` 返回正数后不能直接把整个输出缓冲区清空，只能删除已经发送的 `n` 字节。

8. Reactor 管理全部 Connection

8.1 映射关系

Reactor 至少维护：

```
epoll fd
监听 Connection
fd -> Connection*
连接集合
```

简化结构：

```
class Reactor {
public:
    bool AddConnection(Connection* conn);
    void RemoveConnection(int fd);
    bool EnableReadWrite(
        Connection* conn,
        bool read,
```

```
        bool write
    );
    void Loop();

private:
    int epoll_fd_;
    Connection* listen_conn_;
    std::unordered_map<int, Connection*> connections_;
    bool running_;
};
```

8.2 添加连接

```
bool Reactor::AddConnection(Connection* conn) {
    int fd = conn->SockFd();

    epoll_event ev = {0};
    ev.events = conn->Events();
    ev.data.ptr = conn;

    if (epoll_ctl(
            epoll_fd_,
            EPOLL_CTL_ADD,
            fd,
            &ev) == -1) {
        return false;
    }

    connections_[fd] = conn;
    return true;
}
```

`data.ptr` 直接保存连接对象，事件返回后不必再通过 fd 查二次结构。代价是对象生命周期必须严格管理。

8.3 修改事件

```
bool Reactor::EnableReadWrite(
    Connection* conn,
    bool read,
    bool write
) {
```

```
    uint32_t events = 0;
    if (read) events |= EPOLLIN;
    if (write) events |= EPOLLOUT;

    // 保留默认的 ET 设置或其他连接标志
    events |= (conn->Events() & EPOLLET);

    if (events == conn->Events()) {
        return true;
    }

    epoll_event ev = {0};
    ev.events = events;
    ev.data.ptr = conn;

    if (epoll_ctl(
            epoll_fd_,
            EPOLL_CTL_MOD,
            conn->SockFd(),
            &ev) == -1) {
        return false;
    }

    conn->SetEvents(events);
    return true;
}
```

8.4 删除连接

```
void Reactor::RemoveConnection(int fd) {
    auto it = connections_.find(fd);
    if (it == connections_.end()) return;

    Connection* conn = it->second;

    epoll_ctl(epoll_fd_, EPOLL_CTL_DEL, fd, nullptr);
    connections_.erase(it);

    conn->Close();
    delete conn;
}
```

真实服务器更推荐把 `delete` 延迟到当前事件批次结束，避免仍有回调栈引用该对象。也可以使用引用计数或所有权容器。

9. Reactor 事件循环

```
void Reactor::Loop() {
    epoll_event events[1024];
    running_ = true;

    while (running_) {
        int n = epoll_wait(
            epoll_fd_,
            events,
            1024,
            -1
        );

        if (n < 0) {
            if (errno == EINTR) continue;
            break;
        }

        for (int i = 0; i < n; ++i) {
            auto* conn =
                static_cast<Connection*>(events[i].data.ptr);
            uint32_t mask = events[i].events;

            if (mask & (EPOLLERR | EPOLLHUP)) {
                if (conn && conn->excepter_) {
                    conn->excepter_(conn);
                }
                continue;
            }

            if (mask & EPOLLIN) {
                if (conn && conn->recver_) {
                    conn->recver_(conn);
                }
            }

            if (mask & EPOLLOUT) {
                if (conn && conn->sender_) {
                    conn->sender_(conn);
                }
            }
```

```
        }
    }
}
```

如果回调删除了连接，那么同一个事件的后续分支不能再访问它。实际实现需要“已关闭”标记、延迟回收或安全引用。

10. 板书：Reactor 的层次

板书把 Reactor 放在应用层代码和 epoll 之间：

```
业务 Handler
    |
    v
Connection / Accepter
    |
    v
Reactor
    |
    v
epoll
    |
    v
socket
```

读图时关注三个方向：

10.1 向下：事件从哪里来

epoll 监听 socket 和普通连接，状态变化后把事件交给 Reactor。Reactor 不是自己产生 I/O 事件，而是转发内核通知。

10.2 横向：怎样找到对象

Reactor 通过 `data.ptr` 或 fd 映射找到 `Connection`，并把事件交给该连接的回调。

10.3 向上：状态怎样回到业务

`HandlerConnection` 把字节解释成请求，业务处理后再把响应变成输出缓冲区。发送完成或阻塞时，通过 `EnableReadWrite` 回到 epoll。

11. Connection 生命周期

11.1 创建

```
accept 成功
    -> 创建 Connection
    -> 注册回调
    -> epoll_ctl ADD
    -> 加入 fd map
```

11.2 运行

```
EPOLLIN
    -> Recver
    -> 追加输入缓冲区
    -> Decode/业务/Encode
    -> 追加输出缓冲区
    -> Sender

EPOLLOUT
    -> Sender
    -> 继续发送

EPOLLERR/HUP
    -> Excepter
```

11.3 销毁

```
标记 closing
    -> epoll_ctl DEL
    -> 从 fd map 删除
    -> close socket
    -> 清空缓冲区
    -> 延迟 delete
```

不能在当前 `Sender` 函数尚未返回时立即释放 `conn`，因为后续代码可能仍访问其字段。删除事件和释放对象需要分层完成。

12. 缓冲区上限与背压

Reactor 用内存换取了事件驱动能力，但缓冲区不能无限增长。

风险场景：

- 客户端发送很快，业务处理很慢；
- 服务端产生大量响应，客户端读取很慢；
- 协议声明的消息长度过大；
- 攻击者持续建立连接但不读响应。

需要在连接层设置：

- 输入缓冲区最大长度；
- 输出缓冲区最大长度；
- 单次事件处理上限；
- 空闲连接超时；
- 慢连接淘汰策略。

背压的含义是：当处理能力跟不上时，必须限制上游继续产生数据，而不是让内存无限增长。

13. 业务处理不能阻塞事件循环

直接在事件循环里执行耗时计算，会让同一 Reactor 管理的所有连接等待。典型错误包括：

- 同步等待数据库；
- 执行几秒的复杂计算；
- 阻塞写日志文件；
- 调用可能长时间阻塞的第三方接口。

轻量业务可以在事件循环中完成；重计算应交给工作线程池，完成后通过任务队列通知 EventLoop。

这种跨线程通知正是下一篇 One Thread One Loop 与 eventfd 要解决的问题。